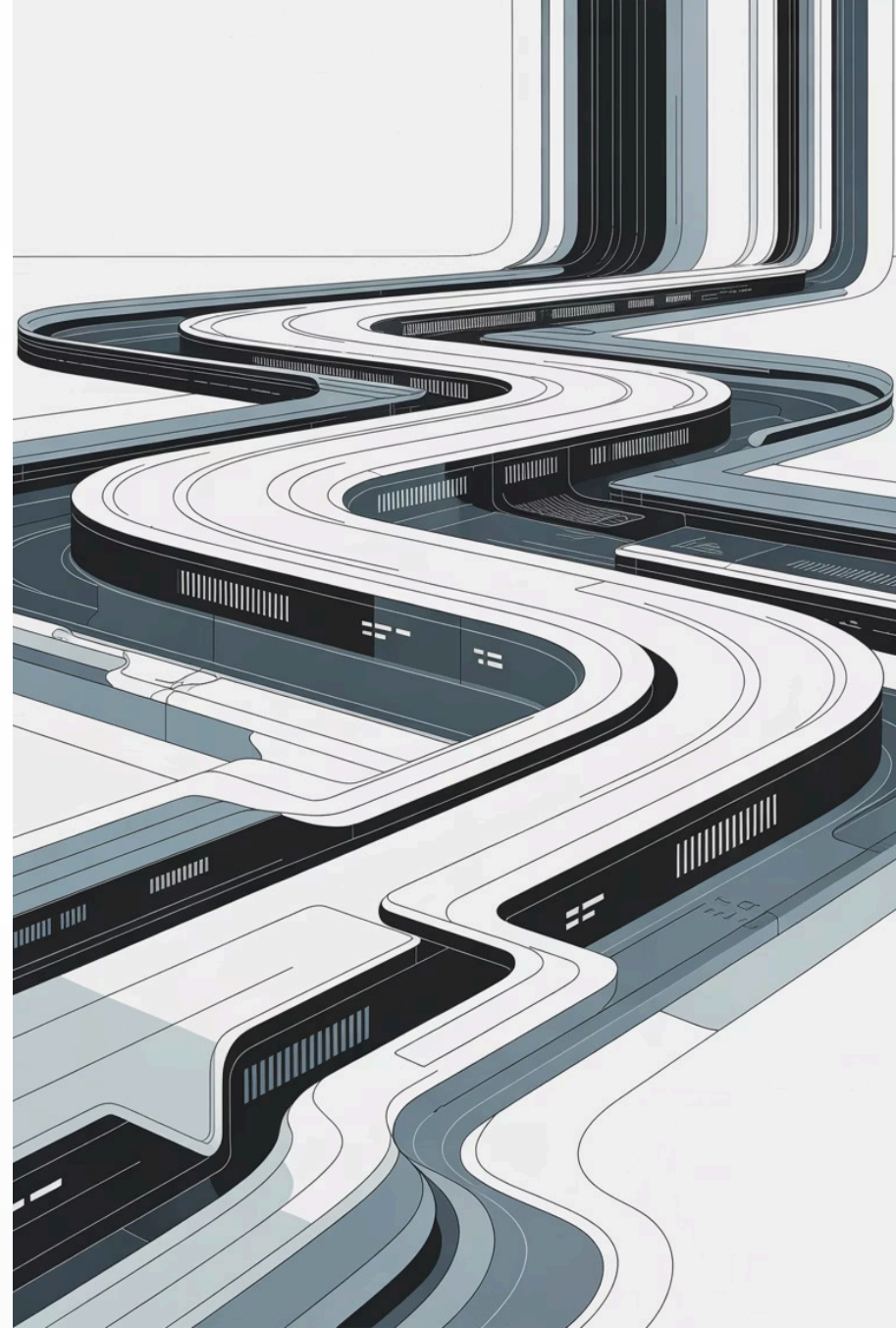


Interfaces Funcionales y Streams en Java

Descubre el poder de la programación funcional en Java y aprende a escribir código más elegante, conciso y expresivo utilizando interfaces funcionales y streams.



Interfaces con Código Ejecutable

Java 8 permite agregar código ejecutable directamente en las interfaces

Desde Java 8 se permite que algunos métodos de las interfaces tengan código. Esto no rompe la idea fundamental de interfaz y las asemeja un poco más a las clases abstractas. Principalmente, existen dos tipos de métodos con implementación en código que se pueden añadir a una interfaz:



Métodos **Default**

Para que las clases tengan una implementación base.

- Permiten añadir nuevos métodos a una interfaz sin romper la compatibilidad de las clases que ya la implementan.
- Útiles para proporcionar un comportamiento por defecto que las clases pueden sobrescribir según sea necesario.



Métodos **Static**

Métodos de utilidad que no dependen del objeto que instancie la interfaz.

- No están asociados a ninguna instancia de la interfaz, son a nivel de la interfaz misma.
- Ideales para funciones auxiliares o de fábrica relacionadas con la interfaz, pero que no requieren el estado de un objeto.

Métodos Default en Interfaces

Con Java 8, las interfaces trascendieron su rol de meras definiciones de contrato, permitiendo la inclusión de código ejecutable directamente **dentro** de ellas, sin afectar la compatibilidad.



Métodos Default

Los métodos **default** permiten añadir nuevas funcionalidades a las interfaces sin romper la compatibilidad con las clases que ya las implementan. Proveen una implementación base directamente **en la interfaz** que las clases pueden usar o sobrescribir.

```
interface Operaciones {  
    int sumar(int a, int b);  
  
    default int restar(int a, int b) { // Implementado en la interfaz  
        return a - b;  
    }  
}  
  
class Calculadora implements Operaciones {  
    @Override  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
    // "restar" es opcional de sobrescribir  
}
```

- **Características clave:** Retrocompatibilidad, implementación por defecto, sobrescribibles.
- **Cuándo usar:** Extender APIs de interfaces sin impactar implementaciones existentes; proporcionar comportamiento común que puede ser reutilizado o especializado.

Métodos Static en Interfaces

Con Java 8, las interfaces trascendieron su rol de meras definiciones de contrato, permitiendo la inclusión de código ejecutable directamente **dentro** de ellas, sin afectar la compatibilidad.



Métodos Static

A diferencia de los métodos `default`, estos métodos pertenecen directamente a la interfaz y no a las instancias de las clases. Funcionan de manera similar a los métodos estáticos en clases, agrupando funcionalidades de utilidad **en la propia interfaz**.

```
interface UtilidadesString {  
    static boolean esVacio(String str) { // Implementado en la interfaz  
        return str == null || str.trim().isEmpty();  
    }  
    static String limpiarEspacios(String str) {  
        return str.trim();  
    }  
}
```

```
// Uso directo de la interfaz:  
// boolean vacio = UtilidadesString.esVacio(" ");
```

- **Características clave:** Pertenencia a la interfaz (se invocan con `Interfaz.metodoStatic()`); no son heredables ni sobrescribibles.
- **Cuándo usar:** Organizar métodos de utilidad relacionados con la interfaz; crear métodos *factory* para producir instancias de las implementaciones de la interfaz.

¿Qué es una Interfaz Funcional?

Una **Interfaz Funcional** (Functional Interface) es una interfaz que contiene **exactamente un único método abstracto**. Representa un comportamiento o función que puede pasarse como argumento, almacenarse en una variable o devolverse desde un método.

Las interfaces funcionales son la base de la programación funcional en Java y permiten usar expresiones lambda y referencias a métodos de forma natural y elegante.



Un método abstracto

Solo puede tener un método sin implementación



Métodos adicionales

Puede incluir métodos `default` y `static`



Anotación opcional

Se recomienda usar `@FunctionalInterface`

Ejemplo de Interfaz Funcional

Definición

```
@FunctionalInterface
interface Operacion {
    int aplicar(int a, int b);
}
```

Esta interfaz define un contrato simple: una operación que toma dos enteros y devuelve un entero como resultado.

Características clave

- **Un único método abstracto:** aplicar(int, int)
- **Anotación @FunctionalInterface:** garantiza que el compilador detecte errores si añadimos más métodos abstractos
- **Reutilizable:** puede implementarse de múltiples formas diferentes

Cuatro Formas de Implementar una Interfaz Funcional

Java ofrece múltiples maneras de implementar interfaces funcionales, desde el enfoque tradicional orientado a objetos hasta las modernas expresiones lambda. Cada forma tiene su lugar y utilidad según el contexto.

01

Clase que implementa la interfaz

Enfoque tradicional con una clase completa

03

Expresión lambda

Sintaxis concisa y moderna introducida en Java 8

02

Clase anónima

Implementación inline sin crear una clase con nombre

04

Referencia a método

Forma abreviada cuando se llama a un método existente

1. Clase que Implementa la Interfaz

```
class Suma implements Operacion {  
    public int aplicar(int a, int b) {  
        return a + b;  
    }  
}
```

```
// Uso  
Operacion op = new Suma();  
int resultado = op.aplicar(5, 3); // 8
```

Cuándo usarla

- Lógica compleja que requiere múltiples métodos auxiliares
- Necesitas reutilizar la implementación en varios lugares
- Quieres mantener estado interno en la clase

2. Clase Anónima

```
Operacion op = new Operacion() {  
    public int aplicar(int a, int b) {  
        return a + b;  
    }  
};
```

```
int resultado = op.aplicar(5, 3); // 8
```

Cuándo usarla

- Implementación única y no reutilizable
- Lógica algo compleja para una lambda
- Código legacy que aún no migró a lambdas



 Las lambdas han reemplazado en gran medida este patrón

3. Expresión Lambda

```
Operacion op = (a, b) -> a + b;
```

```
int resultado = op.aplicar(5, 3); // 8
```

La sintaxis lambda es compacta y expresiva: (parámetros) -> expresión

Ventajas

- **Concisión:** reduce el código ceremonial significativamente
- **Legibilidad:** se centra en la lógica, no en la estructura
- **Expresividad:** comunica la intención de forma clara

Variables Efectivamente Finales en Lambdas

Las expresiones lambda solo pueden usar variables del contexto externo que sean **finales** o **efectivamente finales**. Una variable es efectivamente final si su valor no cambia después de inicializarse, aunque no esté declarada explícitamente como final.

✓ Esto funciona

```
int base = 10; // efectivamente final

Function<Integer, Integer> sumarBase =
    x -> x + base;

System.out.println(sumarBase.apply(5)); // 15
```

La variable `base` nunca cambia, por lo que es efectivamente final.

✗ Esto da error

```
int base = 10;

Function<Integer, Integer> sumarBase =
    x -> x + base;

base = 20; //
```


4. Referencia a Método

```
Operacion op = Integer::sum;  
  
int resultado = op.aplicar(5, 3); // 8
```

Cuando una lambda simplemente llama a un método existente, podemos usar el operador `::` para una sintaxis aún más limpia.

Equivalencia

```
// Lambda  
(a, b) -> Integer.sum(a, b)  
  
// Referencia a método  
Integer::sum
```

Ambas formas son funcionalmente idénticas, pero la referencia a método es más concisa.

Referencias a Métodos: Los Cuatro Tipos

Las referencias a métodos son una forma elegante y legible de referenciar métodos existentes cuando el cuerpo de la lambda únicamente llama a ese método. Existen cuatro tipos según el contexto del método referenciado.



Método estático

`Clase::metodoEstatico`

Ejemplo: `Math::max`



Método de instancia (objeto específico)

`objeto::metodoInstancia`

Ejemplo: `System.out::println`



Método de instancia (objeto arbitrario)

`Clase::metodoInstancia`

Ejemplo: `String::toUpperCase`



Constructor

`Clase::new`

Ejemplo: `ArrayList::new`

Métodos Estáticos: Clase::metodoEstatico

```
import java.util.function.BinaryOperator;

public class DemoEstatico {
    public static int sumar(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        BinaryOperator<Integer> suma =
            DemoEstatico::sumar;
        System.out.println(suma.apply(4, 5)); // 9
    }
}
```

Equivalencia lambda

```
(a, b) -> DemoEstatico.sumar(a, b)
```

Regla fundamental

La signatura del método referenciado debe **coincidir exactamente** con el método abstracto de la interfaz funcional: mismo número de parámetros, mismos tipos y mismo tipo de retorno.

Método de Instancia: Objeto Específico

Concepto

Cuando el método ya está asociado a un **objeto existente**, podemos referenciar ese método específico del objeto.

```
Consumer<String> imprimir = System.out::println;  
imprimir.accept("Hola desde System.out");
```

Aquí, `System.out` es un objeto específico (una instancia de `PrintStream`), y referenciamos su método `println`.

Ejemplo práctico

```
String prefijo = "Hola, ";  
Function<String, String> saludar = prefijo::concat;  
System.out.println(saludar.apply("Juan"));  
// "Hola, Juan"
```

Equivalencia lambda

```
s -> System.out.println(s)  
s -> prefijo.concat(s)
```

Método de Instancia: Objeto Arbitrario

Este tipo es especialmente útil en streams. El método se aplicará a **cada objeto** del tipo indicado. Java infiere que el primer argumento será el objeto receptor.

```
List<String> nombres = List.of("ana", "juan", "luis");
nombres.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

```
// Output:
// ANA
// JUAN
// LUIS
```

Equivalencia lambda

```
.map(s -> s.toUpperCase())
```

¿Cómo funciona?

Cada elemento del stream (un String) se convierte en el objeto receptor del método `toUpperCase()`.

Referencias a Constructores

Sintaxis: Clase::new

```
Supplier<List<String>> crearLista = ArrayList::new;  
List<String> lista = crearLista.get();
```

Permite crear objetos nuevos sin escribir explícitamente `new` dentro de la lambda.

Equivalencia lambda

```
() -> new ArrayList<>()
```

Uso común en streams

```
List<String> nombres = stream  
    .collect(Collectors.toCollection(ArrayList::new));
```

Implementaciones Implícitas de Interfaces Funcionales

Una característica fascinante del modelo funcional de Java es que **no es necesario declarar explícitamente** que una clase implementa una interfaz funcional. Si la firma del método coincide, Java permite usarlo como implementación.



Type Matching Estructural

Java compara las firmas de métodos, no solo las relaciones nominales (implements)



Mayor Flexibilidad

Métodos compatibles pueden usarse aunque la clase no implemente la interfaz



Base de las Lambdas

Este mecanismo permite que las lambdas funcionen sin declaraciones explícitas

Ejemplo de Implementación Implícita

```
@FunctionalInterface
interface Validador {
    boolean validar(String texto);
}

class ValidadorTexto {
    public boolean validar(String texto) {
        return texto != null && !texto.isEmpty();
    }
}

public class Demo {
    public static void main(String[] args) {
        ValidadorTexto vt = new ValidadorTexto();
        //
```


Interfaces Funcionales Predefinidas en Java

El paquete `java.util.function` proporciona un conjunto completo de interfaces funcionales listas para usar, cubriendo los casos más comunes en programación funcional.

Interfaz	Método	Descripción	Ejemplo
<code>Predicate<T></code>	<code>boolean test(T t)</code>	Evalúa una condición y devuelve verdadero o falso	<code>x -> x > 0</code>
<code>Function<T,R></code>	<code>R apply(T t)</code>	Transforma un valor de tipo T en tipo R	<code>x -> x * 2</code>
<code>Consumer<T></code>	<code>void accept(T t)</code>	Realiza una acción sin devolver nada	<code>System.out::println</code>
<code>Supplier<T></code>	<code>T get()</code>	Suministra un valor sin recibir argumentos	<code>() -> Math.random()</code>
<code>UnaryOperator<T></code>	<code>T apply(T t)</code>	Function especializada: entrada y salida del mismo tipo	<code>x -> x + 1</code>
<code>BinaryOperator<T></code>	<code>T apply(T a, T b)</code>	Function con dos argumentos del mismo tipo	<code>(a,b) -> a + b</code>

Streams en Java: Procesamiento Declarativo

Un **Stream** es una secuencia de elementos sobre la que se pueden aplicar operaciones funcionales de forma declarativa. Los streams permiten expresar transformaciones complejas de datos de manera clara y concisa, sin modificar las colecciones originales.

No almacenan datos

Los streams operan sobre la fuente original sin copiar elementos

Operaciones funcionales

filter, map, reduce, collect y muchas más

Evaluación perezosa

Las operaciones intermedias solo se ejecutan cuando hay una operación terminal

Pipeline de operaciones

Se encadenan múltiples transformaciones de forma fluida

Ejemplo Básico de Stream

```
List<String> nombres = List.of("Ana", "Luis", "Juan");
```

```
nombres.stream()  
    .filter(n -> n.length() > 3)  
    .map(String::toUpperCase)  
    .forEach(System.out::println);
```

```
// Output:
```

```
// LUIS
```

```
// JUAN
```

Flujo de operaciones

1. **Fuente:** nombres.stream() crea el stream
2. **Filtrado:** mantiene solo nombres con más de 3 caracteres
3. **Transformación:** convierte a mayúsculas
4. **Acción:** imprime cada elemento

Las Tres Fases de un Stream

1. Fuente de Datos

Origen del stream: colecciones, arrays, ficheros, generadores

```
list.stream()  
Stream.of(1, 2, 3)  
Files.lines(path)
```

2. Operaciones Intermedias

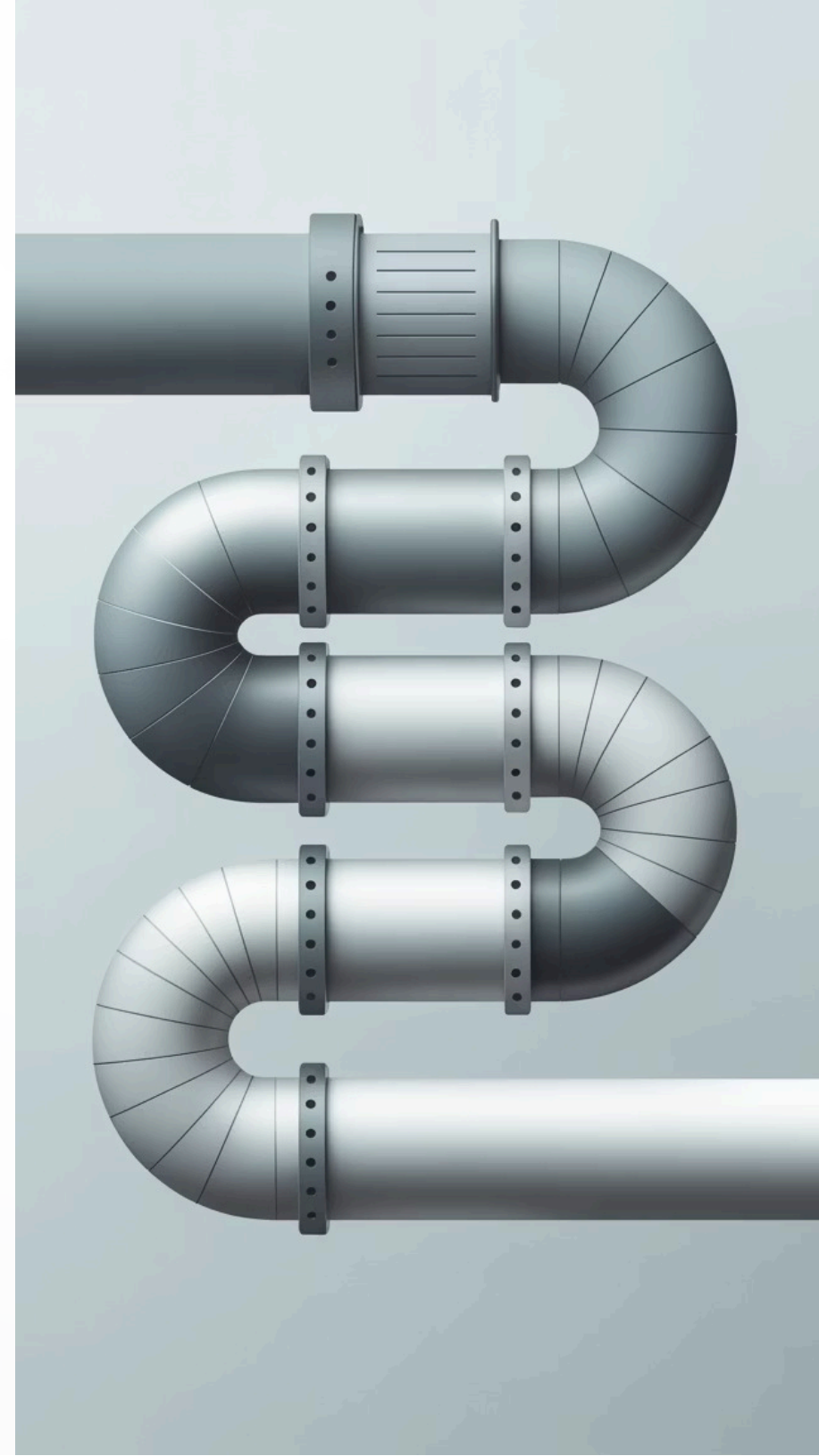
Transformaciones que devuelven otro stream. Son **perezosas**: no se ejecutan hasta que hay una operación terminal

```
filter(), map(), sorted()  
distinct(), limit()
```

3. Operación Terminal

Produce un resultado final o efecto secundario. Activa la ejecución del pipeline

```
forEach(), collect()  
reduce(), count(), findFirst()
```



Interfaces Funcionales en Métodos de Stream

Cada método de stream utiliza una interfaz funcional específica según el tipo de operación que realiza. Comprender estas relaciones facilita el uso efectivo de streams.

Método Stream	Interfaz Funcional	Ejemplo
filter(Predicate)	Predicate<T>	stream.filter(x -> x > 5)
map(Function)	Function<T,R>	stream.map(String::length)
forEach(Consumer)	Consumer<T>	stream.forEach(System.out::println)
reduce(BinaryOperator)	BinaryOperator<T>	stream.reduce((a,b) -> a+b)
sorted(Comparator)	Comparator<T>	stream.sorted((a,b) -> a.compareTo(b))

El Método reduce(): Concepto Fundamental

El método `reduce()` es una operación terminal que combina todos los elementos del stream en un **único resultado**, aplicando repetidamente una función acumuladora. Su nombre proviene de "reducir" una secuencia completa a un solo valor.



Dos formas de usar reduce()

1

Sin valor inicial

```
List<Integer> numeros = List.of(2, 3, 4);

Optional<Integer> resultado = numeros.stream()
    .reduce((a, b) -> a * b);

resultado.ifPresent(System.out::println); // 24
```

Devuelve **Optional** porque el stream podría estar vacío. El primer elemento actúa como valor inicial.

2

Con valor inicial (identidad)

```
List<Integer> numeros = List.of(2, 3, 4);

int producto = numeros.stream()
    .reduce(1, (a, b) -> a * b);

System.out.println(producto); // 24
```

Devuelve directamente el tipo del resultado, nunca null. El valor inicial garantiza que siempre hay resultado.

Optional: Manejo Seguro de Valores Nulos

Introducido en Java 8 para evitar `NullPointerException` de forma elegante

La interfaz `Optional` en Java es un contenedor que puede o no contener un valor no nulo. Su propósito principal es permitirle trabajar con valores posiblemente nulos de una manera más segura y expresiva, reduciendo la aparición de las temidas `NullPointerException` y mejorando la legibilidad del código. Es una herramienta poderosa para indicar la ausencia de un valor, obligando al desarrollador a considerar estos casos.

	
Crear Optional <pre>String nombre = "Juan"; Optional<String> optNombre = Optional.of(nombre); // Optional[Juan] // Optional.of(null); // Lanza NullPointerException</pre><pre>String apellido = "Perez"; Optional<String> optApellido = Optional.ofNullable(apellido); // Optional[Perez]</pre><pre>String segundoNombre = null; Optional<String> optSegundoNombre = Optional.ofNullable(segundoNombre); // Optional.empty</pre><pre>Optional<String> optVacio = Optional.empty(); // Optional.empty</pre> • Optional.of(valor): Crea un <code>Optional</code> que contiene el valor especificado. Lanza <code>NullPointerException</code> si el valor es <code>null</code>. Úselo cuando esté seguro de que el valor no es nulo. • Optional.ofNullable(valor): Crea un <code>Optional</code> que contiene el valor especificado si no es nulo, o un <code>Optional</code> vacío si el valor es nulo. Úselo cuando el valor pueda ser nulo. • Optional.empty(): Crea un <code>Optional</code> vacío explícitamente. Útil cuando se necesita representar la ausencia de un valor de forma directa.	Usar Optional <pre>Optional<String> optValor = Optional.of("Hola"); if (optValor.isPresent()) { System.out.println("Valor: " + optValor.get()); // No hacer .get() sin isPresent() } // También: optValor.isEmpty()</pre><pre>optValor.ifPresent(v -> System.out.println("Valor existe: " + v));</pre><pre>String result = optVacio.orElse("Default"); // "Default"</pre><pre>String resultLazy = optVacio.orElseGet(() -> "Calculado por defecto"); // "Calculado por defecto"</pre><pre>Optional<Integer> longitud = optValor.map(String::length); // Optional[4]</pre><pre>Optional<String> filtrado = optValor.filter(s -> s.startsWith("H")); // Optional[Hola] Optional<String> noFiltrado = optValor.filter(s -> s.startsWith("Z")); // Optional.empty</pre> • isPresent() / isEmpty(): Verifica si el <code>Optional</code> contiene un valor (<code>isPresent()</code>) o si está vacío (<code>isEmpty()</code>). • ifPresent(Consumer): Ejecuta una acción si el <code>Optional</code> contiene un valor. • orElse(valor): Devuelve el valor contenido si existe, o un valor por defecto especificado si el <code>Optional</code> está vacío. • orElseGet(Supplier): Similar a <code>orElse</code>, pero el valor por defecto se evalúa de forma lazy (solo si el <code>Optional</code> está vacío), lo que puede ser más eficiente si la creación del valor por defecto es costosa. • map(Function): Si hay un valor, aplica una función de mapeo al valor y devuelve un nuevo <code>Optional</code> con el resultado. Permite encadenar operaciones. • filter(Predicate): Si hay un valor y cumple una condición, devuelve el mismo <code>Optional</code>. De lo contrario, devuelve un <code>Optional</code> vacío. Estos métodos permiten un manejo seguro y funcional de la ausencia de valores, eliminando la necesidad de verificaciones <code>null</code> explícitas y las potenciales <code>NullPointerException</code> asociadas.

En resumen, `Optional` ayuda a escribir un código **más seguro** al hacer explícita la posibilidad de ausencia de un valor, **más legible** al usar un flujo de trabajo funcional y encadenado, y **más expresivo** al comunicar claramente las intenciones del desarrollador.

Ejemplo Práctico: Concatenación de Cadenas

Uso de reduce() con Strings

```
List<String> nombres =  
    List.of("Ana", "Luis", "Juan");  
  
String resultado = nombres.stream()  
    .reduce("", (a, b) -> a + " " + b);  
  
System.out.println(resultado.trim());  
// "Ana Luis Juan"
```

Paso a paso

1. **Inicio:** acc = ""
2. **Iteración 1:** "" + " " + "Ana" = " Ana"
3. **Iteración 2:** " Ana" + " " + "Luis" = " Ana Luis"
4. **Iteración 3:** " Ana Luis" + " " + "Juan" = " Ana Luis Juan"
5. **Final:** aplicamos trim() para eliminar espacios

Ejemplo Completo: Combinando Todo

```
List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6);

int resultado = numeros.stream()
    .filter(n -> n % 2 == 0)    // Predicate: filtra pares
    .map(n -> n * n)           // Function: eleva al cuadrado
    .reduce(0, (a, b) -> a + b); // BinaryOperator: suma total

System.out.println(resultado); // 56
```

1

1. Filtrado

Mantiene solo números pares

[2, 4, 6]

2

2. Transformación

Eleva cada número al cuadrado

[4, 16, 36]

3

3. Reducción

Suma todos los valores

$4 + 16 + 36 = 56$

Este ejemplo demuestra el poder de la programación funcional: expresamos transformaciones complejas de datos de forma declarativa, clara y concisa, encadenando operaciones que se leen como un flujo natural de procesamiento.